

Context Foundry: Research Request for Strategic Direction

Who We Need Help From

We need deep research and strategic thinking from an AI system that can:

Understand the full problem space

Identify who has solved similar problems

Recommend how to navigate competing constraints

Help us define the right success metrics

What Is Context Foundry?

Context Foundry (CF) is an enterprise knowledge system that answers questions about an organization's technical infrastructure using a knowledge graph. Think of it as "an AI that knows your company's systems and can answer questions about them."

Example queries:

"What services depend on the Payment Database?"

"What's the blast radius if the Auth Service goes down?"

"Who owns the Notification Service?"

The core promise: CF provides "organizational object permanence for AI" — meaning the AI remembers what it has been told about the organization and doesn't make things up.

Target users: Enterprise decision-makers (VPs, SREs, architects) who need reliable answers about their systems.

The Goal We're Trying to Reach

Primary goal: Provide useful, accurate, grounded answers to enterprise queries about technical infrastructure.

This breaks down into:

Useful — The answer actually helps the person make a decision or understand something

Accurate — The facts stated are correct

Grounded — Every claim is traceable to data in the knowledge graph

Timely — Answers come in reasonable time (seconds, not minutes)

The constraint: Don't hallucinate. If CF doesn't have data, it should say so rather than invent plausible-sounding but false information.

Critical distinction: "Don't hallucinate" is a CONSTRAINT, not the OBJECTIVE. A system that says "I don't know" to everything satisfies the constraint but fails the objective.

The Journey So Far

Phase 1: Initial Build

Built a knowledge graph system with:

Semantic Memory: Entities (services, databases, teams) and relationships (DEPENDS_ON, OWNS, etc.)

Episodic Memory: Incident history, events, changes

Symbolic Memory: Rules, policies, escalation paths

Created a retrieval-augmented generation (RAG) pipeline:

User asks a question

System retrieves relevant entities and relationships from the graph

LLM reasons over the retrieved context

LLM produces an answer

Phase 2: Data Pipeline Problem

Problem discovered: Only 328 of 8,500+ entities were queryable. Most entities were stuck in a staging state and never promoted to the queryable "TRUSTED" state.

Solution: Fixed the pipeline to promote entities. Result: 8,511 entities now queryable.

New problem emerged: Hallucination rate spiked from 1% to 26.7%.

Phase 3: Hallucination Crisis

Root cause analysis:

We had 8,511 entities but only 2,013 relationships

That's ~0.24 relationships per entity

~6,500 entities exist but have ZERO relationships

When the LLM sees an entity exists but has no relationship data, it invents "plausible" relationships

Example hallucination:

User asks: "What depends on Payment Gateway?"

Payment Gateway exists in the graph but has no DEPENDS_ON relationships documented

LLM invents: "Auth Service and Checkout Service likely depend on Payment Gateway"

This is false — we have no evidence for this

Phase 4: The "Data Gates" Fix

We consulted four independent AI systems (Perplexity, Gemini, Claude, ChatGPT) about how to solve the hallucination problem. They unanimously agreed:

"Don't ask the LLM to not hallucinate. Don't give it the opportunity to hallucinate."

Solution implemented: Add architectural "gates" that detect insufficient data BEFORE calling the LLM:

Entity Guard: If entity doesn't exist → return "Entity not found"

Relationship Guard: If entity exists but has 0 relationships AND query is about relationships → return "Entity exists but has no documented relationships" (DON'T call LLM)

Sufficiency Check: If entity has 1-2 relationships → simple formatting only (limited LLM reasoning)

Constrained Prompt: For entities with adequate data → LLM must cite sources, no speculation allowed

Results:

| Metric | Before Gates | After Gates |

|-----|-----|-----|

| Hallucination | 26.7% | 0% |

| GATED (refused) | N/A | 40% |

| NOT_FOUND | 0% | 10% |

| ACCURATE | 26.7% | 10% |

Phase 5: Current Realization

The problem: We optimized for "zero hallucination" so aggressively that we broke usefulness.

40% of queries now get "I don't have data for this" — that's not helpful

Only 10% of queries get actually useful answers

Entity resolution is buggy (matched "Party" when user asked about "payment processor")

Response times vary wildly (3 seconds to 31 seconds)

We treated the constraint as the goal. A system with 0% hallucination and 10% usefulness has failed.

Current State: What's Working and What's Broken

What's Working

Honesty: The system no longer lies. When it doesn't know, it says so.

Citation: When the system does answer, it cites specific relationships by ID.

Architecture: The guard system is deterministic and reliable.

Coverage: 8,511 entities are now in the graph (vs 328 before).

What's Broken

Usefulness: Only 10% of queries get actually helpful answers.

Entity Resolution: The system sometimes matches the wrong entity (e.g., "Party" instead of "payment processor").

Data Sparsity: 0.24 relationships per entity means most entities are isolated nodes.

Guard Aggressiveness: 40% GATED rate means users hit walls constantly.

Speed Inconsistency: Response times range from 3 seconds to 31 seconds.

User Experience: Being told "no data" repeatedly erodes trust in the system's value.

Example Failures

Query: "What is the impact of losing connectivity to the third-party payment processor?"

What happened: System matched entity "Party" (a person) instead of understanding "third-party payment processor" as a concept. Returned "no relationships for Party."

What should have happened: System should have either (a) found the relevant payment processor entity, or (b) understood the query conceptually and searched for payment-related services, or (c) asked for clarification.

The Core Tension

We're caught between competing goods:

If we prioritize...

We risk...

Accuracy

Being unhelpful (saying "I don't know" too often)

Helpfulness

Hallucinating (inventing plausible but false information)

Speed

Shallow answers (not enough reasoning)

Depth

Slow responses (31 seconds is too long)

The current system chose accuracy over helpfulness. We eliminated hallucination but also eliminated usefulness.

Questions We Need Answered

1. Problem Definition

What is the RIGHT way to frame this problem?

Are we building a "question-answering system" or something else?

What should the success metric actually be?

2. Prior Art

Who else has built enterprise knowledge graph Q&A systems?

How do they handle the sparse data problem?

How do they balance accuracy vs helpfulness?

What does "good" look like in this space?

3. Entity Resolution

How should the system handle queries that don't match entity names exactly?

Should it use semantic search? Fuzzy matching? Query expansion?

How do systems like Google, Siri, or Alexa handle entity disambiguation?

4. Sparse Graph Strategies

When an entity has no relationships, what should the system do?

Are there ways to provide value without hallucinating?

Can episodic memory (incidents) fill gaps when semantic memory (relationships) is empty?

Should the system suggest what data is MISSING rather than just saying "no data"?

5. User Experience

What do enterprise users actually need from a system like this?

Is "I don't know" ever acceptable? When?

How do other enterprise AI tools handle uncertainty?

6. Guard Calibration

Are our guards too aggressive?

What's the right threshold for "insufficient data"?

Should 0 relationships trigger a hard gate, or should the system try harder?

7. Architecture

Is RAG the right pattern for this use case?

Should we consider other approaches (graph neural networks, structured query generation, hybrid systems)?

How do production systems like Glean, Guru, or Notion AI handle this?

What We've Tried

Approach

Result

Problem

Prompt engineering ("don't hallucinate")

Failed

LLM ignored instructions

Negative instructions ("do NOT infer")

Failed

LLM trained to be helpful overrides this

Positive instructions ("ONLY state facts from context")

Partial success

Still hallucinated when context was empty

Hard gates (refuse if 0 relationships)

Eliminated hallucination

Killed usefulness (40% refusal rate)

Citation requirements

Works

Adds overhead, sometimes awkward

GROUNDED/GAPS format

Works

But "GAPS" section doesn't help user

Constraints We Must Honor

No hallucination: We cannot ship a system that invents false information about enterprise infrastructure. Wrong information could lead to bad decisions during outages.

Grounded claims: Every factual claim must be traceable to the knowledge graph. "Auth Service depends on User Database" must correspond to an actual relationship in the graph.

Enterprise-grade: This is for VPs and SREs making real decisions. It must be reliable, professional, and trustworthy.

Reasonable latency: Answers should come in <10 seconds for most queries.

What We're Asking For

Reframe the problem: Help us understand what we're actually building and what success looks like.

Find prior art: Who has solved this? What can we learn from them?

Recommend an approach: Given the constraints, what's the best path forward?

Define metrics: What should we actually measure to know if we're succeeding?

Prioritize: What should we fix first? Entity resolution? Guard calibration? Data enrichment?

Something else?

Summary

We built a system that doesn't lie but also doesn't help.

The constraint (don't hallucinate) consumed the objective (be useful). We need help finding the balance — a system that is both honest AND helpful, that knows when to answer and when to ask for clarification, that provides value even when data is incomplete.

This is not a novel problem. Enterprise knowledge systems, search engines, virtual assistants — they all face this tension. Someone has solved it before. We need to learn from them.

Help us find the way.